



CMPT 295

Unit - Machine-Level Programming

Lecture 22 – Assembly language – Arrays

Last Lecture

- **x86-64 register saving convention**

Responsibility of **callee**

1. **callee** saved registers:

- **Callee** must save before using register & must restore register before control returns to **caller**

% <u>rbx</u>
%r12
%r13
%r14
%r15
% <u>rbp</u>
% <u>rsp</u>

Responsibility of **caller**

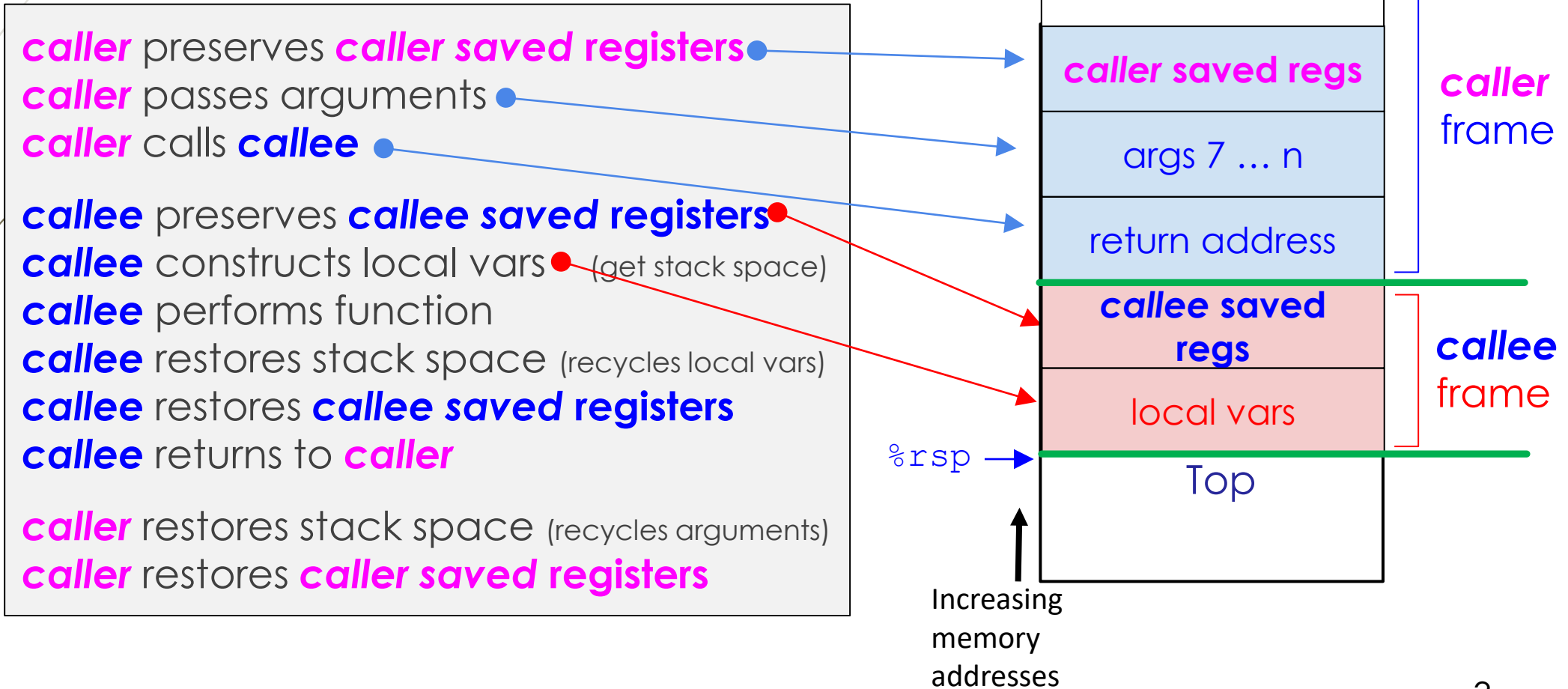
2. **caller** saved registers:

- **Caller** must save register before calling **callee** & must restore register after control has returned to **caller**

Return value	% <u>rax</u>
Parameters/ arguments	% <u>rdi</u>
	% <u>rsi</u>
	% <u>rdx</u>
	% <u>rcx</u>
	%r8
	%r9
	%r10
	%r11

Last lecture

- x86-64 conventions and stack frame



Observation: caller is a callee and callee can become a caller!

Last Lecture

- Recursion
 - Handled without special instruction
 - Using stack frames
 - Using x86-64 function call and Register saving conventions

Today's Menu

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point operations

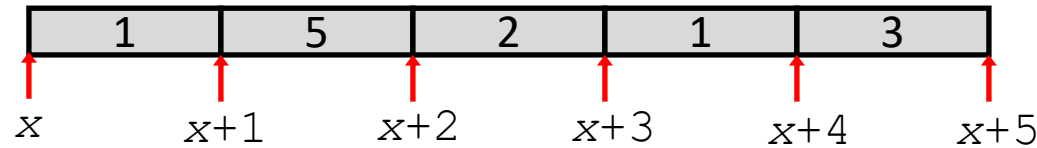
Review - 1D Array – in C

T **$A[n]$** ;

A is an array of data type **T** with **n** cells

A is a contiguously allocated memory space of length **$n * L$** bytes where **$L = \text{sizeof}(T)$**

For example: `int x[5];`



➡ In C, arrays are also pointers!

Expressed in C

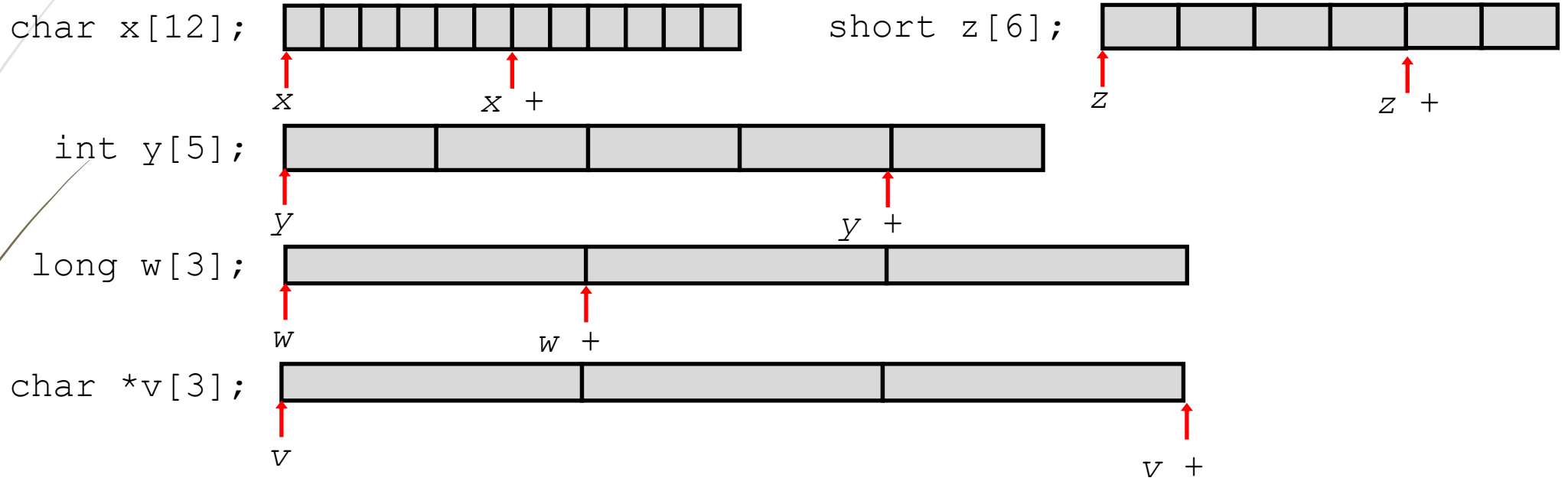
I want ...	Type	Value of result
<code>x[4]</code>	<code>int</code>	
<code>x</code>	<code>int *</code>	
<code>x + 1</code>	<code>int *</code>	
<code>&x[2]</code>	<code>int *</code>	
<code>x[5]</code>	<code>int</code>	
<code>*(x + 1)</code>	<code>int</code>	
<code>x + i</code>	<code>int *</code>	

1D Array – in x86-64

- There are no arrays!
- However, because arrays are contiguously allocated memory spaces, we access an array element by adding a *displacement* to the *memory address of the first byte* of this memory space
 - Use *index arithmetic* to compute memory address of each array element
 - Memory address of array element ($A[i]$) $\Rightarrow A + i * L$
 - where A is *base memory address*
 - i.e., A is the memory address of array element at index 0 ($i=0$)
 - where i is *index of array element*
 - where L is *sizeof(T)*
 - $\Rightarrow$ We can access array elements by adding *displacement* $i * L$ to *memory address of the first byte of memory space allocated to A*

Accessing 1D Array – in x86-64

Arrays
declared
using
C syntax



Manipulating 1D array – Example - main.c

```
#include <stdio.h>
#include <stdlib.h>

char sumChar(char *, int);
short sumShort(short *, int);
int sumInt(int *, int);
long sumLong(long *, int);
```

```
#define N 6
```

Test cases

```
char AC[N] = {-58, 22, 101, -15, 72, 27};
short AS[N] = {-58, 22, 101, -15, 72, 27};
int AI[N] = {258, 522, 1010, -15, -3372, 27};
long AL[N] = {258, 522, 1010, -15, -3372, 27};
```

Expected results

```
// Expected results: sum = -107
// Expected results: sum = 149
// Expected results: sum = -1570
// Expected results: sum = -1570
```

```
void main () {
```

```
    printf("The total of AC is %hi.\n", sumChar(AC, N));
    printf("The total of AS is %hi.\n", sumShort(AS, N));
    printf("The total of AI is %d.\n", sumInt(AI, N));
    printf("The total of AL is %ld.\n", sumLong(AL, N));
    return;
```

```
}
```

This program defines 4 arrays:

- an array of chars,
- an array of shorts,
- an array of ints,
- an array of longs

then it calls the appropriate sum function, i.e., the one that sums elements of the correct data type.

Manipulating 1D array – Example - sums.s - Part 1

- Register **%rdi** contains starting address of array (base address of array)
- Register **%esi** contains size of array (N)
- Register **%ecx** contains array index
- Register **%al** or **%ax** contains the running sum

```
.globl sumChar
sumChar:
    movl    $0,    %eax
    movl    $0,    %ecx
loopChar:
    cmpl    %ecx,   %esi
    jle     endloop1
    addb    (%rdi,%rcx,1), %al
    incl    %ecx
    jmp     loopChar
endloop1:
    ret
```

```
.globl sumShort
sumShort:
    xorl    %eax,   %eax
    xorl    %ecx,   %ecx
    jmp     cond1
loopShort:
    addw    (%rdi, %rcx, 2), %ax
    incl    %ecx
cond1:
    cmpl    %esi,   %ecx
    jne     loopShort
    ret
```

Manipulating 1D array – Example - sums.s - Part 2

- Register **%rdi** contains starting address of array (base address of array)
- Register **%esi** contains size of array (N)
- Register **%ecx** contains array index
- Register **%eax** or **%rax** contains the running sum

```
.globl  sumInt
sumInt:
    xorl    %eax, %eax
    xorl    %ecx, %ecx
    jmp     cond2
loopInt:
    addl    (%rdi, %rcx, 4), %eax
    incl    %ecx
cond2:
    cmpl    %esi, %ecx
    jne     loopInt
    ret
```

```
.globl  sumLong
sumLong:
    movq    $0, %rax
    movl    $0, %ecx
loopLong:
    cmpl    %ecx, %esi
    jle     endloop
    addq    (%rdi, %rcx, 8), %rax
    incl    %ecx
    jmp     loopLong
endloop:
    ret
```

2D Array – array of arrays

T **A**[**R**][**C**];

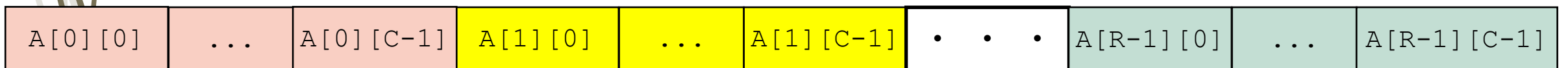
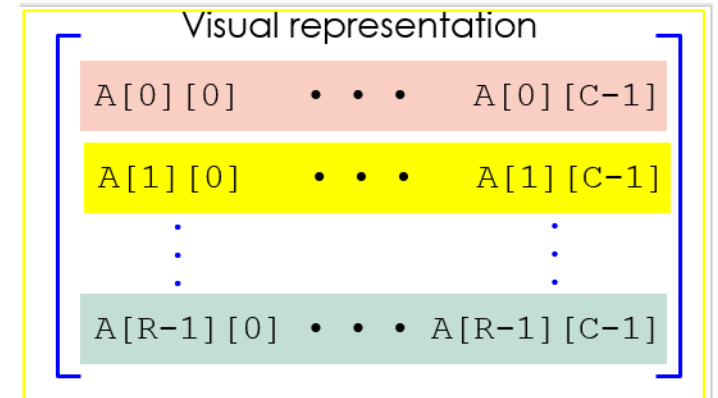
➤ **in C**: declaration for a 2D array of data type **T** with **R** rows and **C** columns

➤ **in memory**:

➤ This array **A** is a contiguously allocated space of **R * C * L** bytes where **L = sizeof(T)** in bytes

➤ Array **A** layout -> **Row-Major** ordering

➤ Example using **int A**[**R**][**C**];

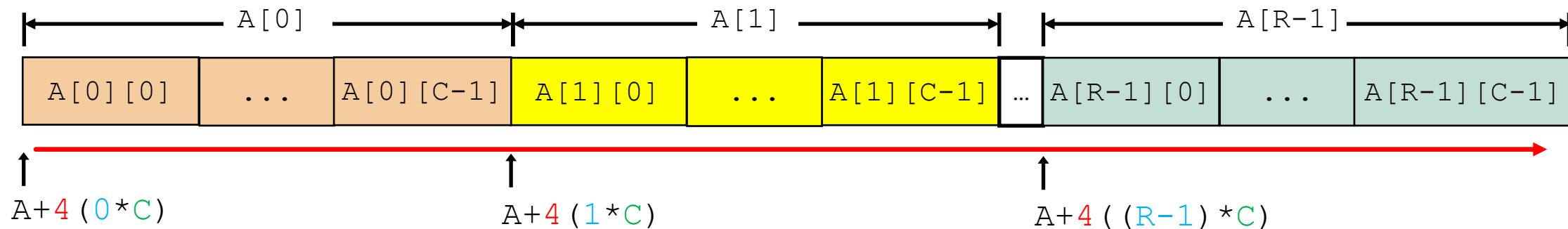


R * C * # of Bytes

Accessing a row of 2D array

T **A**[**R**][**C**];

- **A**[**i**] is an array of **C** elements (row **i** of array **A**)
- Memory address of each row **A**[**i**]: $\mathbf{A} + \mathbf{L} * (\mathbf{i} * \mathbf{C})$
 - where **A** is **base memory address**
 - where **i** is row **index** of the specific row we wish to access
 - where **C** is the number of elements per row
 - where **L** is **sizeof(T)** in bytes
- Can access other rows by incrementing **A** by **L** (**i** * **C**)
- Example using **int A**[**R**][**C**];



Accessing an element of 2D array

T **A**[**R**][**C**];

➤ **A**[**i**][**j**] is an element of type **T**, which requires **L** bytes

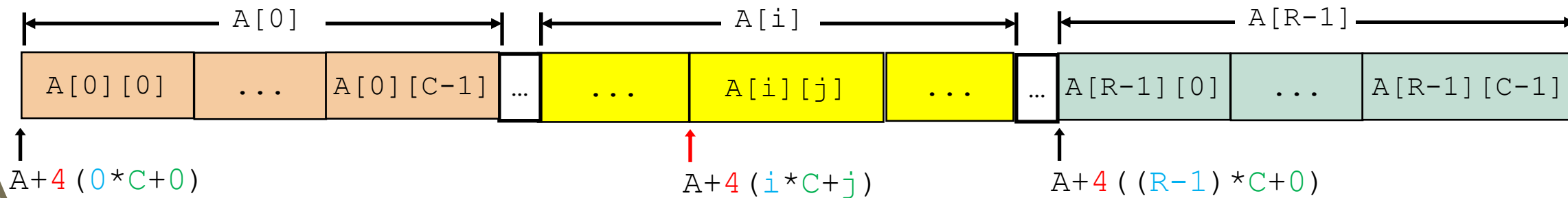
➤ Memory address of each element **A**[**i**][**j**]:

$$\mathbf{A} + \mathbf{L}(\mathbf{i} * \mathbf{C}) + \mathbf{L}(\mathbf{j}) = \mathbf{A} + \mathbf{L}(\mathbf{i} * \mathbf{C} + \mathbf{j})$$

➤ where **i** is row **index** of the specific element we wish to access

➤ where **j** is column (cell) **index** of the specific element we wish to access

➤ Example using **int A**[**R**][**C**];



Example: `short A[3][5];`

➡ In memory:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----

1) Let's compute the memory address to access:

➡ Accessing 3rd row `A[2]` :

➡ Accessing cell `A[2][3]` :

Demo - Accessing an element of 2D array

```
#define N 4

char A[N][N] = { 1, -2, 3, -4,
                 -5, 6, -7, 8,
                 -1, 2, -3, 4,
                 5, -6, 7, -8};

char C[N][N];
```

```
void main() {
    int i, j;

    printf("Matrix A: \n");
    printMatrixByRow(A, N);

    printf("Copy A to C\n");
    copy(A, C, N);

    printf("\nMatrix C: \n");
    printMatrixByRow(C, N);

    return;
}
```

+ **copy.s** on our course web site



Warning!

- In C: To go from one array element to the next, we *increment the index by 1*
- In x86-64: To go from one array element to the next, we add a **displacement** to **the base memory address** which *takes into consideration the size of the data*
- Forgetting that the memory addresses of contiguous array cells differ by **the size of the data** (i.e., the content of the array cells) instead of by 1 is **a common mistake in assembly language programming**

Example of Spilling onto the Stack

- When manipulating data (like array elements), it is time efficient to make use of registers
- If all registers are used OR we want to reuse a register like `rax`, then we use the stack -> called **spill** to stack
- Example: Perform dot product of two `char` matrices A and B (2x2) -> Here, only showing: $A_{00} * B_{00} + A_{01} * B_{10}$

dot:

```
# A in %rdi, B in %rsi, dot product in %rax
movq (%rdi), %rax      # store A00 in %rax
imul (%rsi), %rax      # A00*B00
pushq %rax             # store partial result on stack
movq 1(%rdi), %rax     # overwrite %rax with A01
imul 2(%rsi), %rax     # A01*B10
addq (%rsp), %rax      # A01*B10 + A00*B00
leal 8(%rsp), %rsp     # restore %rsp (same as addq $8, %rsp)
# note: cannot "popq %rax" - why?
```

Summary

➤ Manipulation of 1D arrays – in x86-64

- From x86-64's perspective, a 1D array is a contiguously allocated memory space of $n * L$ bytes where n is the size of the array, i.e., the number of cells where $L = \text{sizeof}(T)$ and T is the data type of the elements stored in the array
- To compute the memory address of each array element

$$\text{➤ } A[i] = \underbrace{A}_{\substack{\text{Base} \\ \text{memory} \\ \text{address}}} + \underbrace{i * L}_{\text{Displacement}}$$

where i is the index of the array element

Summary

➤ Manipulation of 2D arrays – in x86-64

- From x86-64's perspective, a 2D array is a contiguously allocated memory space of $R * C * L$ bytes where R is the height (rows) and C is the width (cols) where $L = \text{sizeof}(T)$ and T is the data type of the elements stored in the array

- 2D array layout in memory: Row-Major ordering

- To compute the memory address of each row

$$A[i] : A + L(i * C)$$

- To compute the memory address of each element

$$A[i][j] : A + L(i * C + j)$$

Base memory address Displacement

Next Lecture

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point operations